STREAMIFY

A Modern Peer-to-Peer Skill-Sharing & Language Exchange Social Network

Preliminary Project Defense Documentation

Prepared By:

Mohamed Ahmed Zaghlol

Project Repository:

`https://github.com/mohamedahmedzaghlol/Streamify-Language-Exchange.git`

Table of Contents

1. Abstract

Modern social software frameworks consistently emphasize generic multi-directional media dissemination rather than structured knowledge allocation. This project presents **Streamify**, an advanced web application explicitly tailored for collaborative bilateral learning nodes, skill matching, and real-time structured socialization. Engineered over the MERN technology stack—incorporating MongoDB, Express.js, React.js, and Node.js—the software addresses systemic gaps in distributed mutual learning environments.

The backend engine utilizes Mongoose object modeling to isolate volatile relational logic, safeguarding low latency during heavy querying actions. By offloading chat socket operations to the cryptographically secure infrastructure of Stream.io SDK via programmatic token provisioning, Streamify ensures data security and real-time stability.

This documentation provides an expansive breakdown of the preliminary lifecycle components. It details structural problem framing, database optimization vectors, granular application routes, and integration validation schemas verified extensively through systematic automated Postman integration pipelines. The output serves as a clear blueprint for deployment onto scalable cloud execution nodes.

2. Introduction

2.1 Project Background

The democratization of specialized technical engineering knowledge and linguistic fluency requires accessible, zero-cost interaction channels. While digital education systems have grown significantly, they remain predominantly one-directional. Users consume pre-recorded video components or articles without contextual or practical peer-to-peer applications.

Bilateral peer tutoring has proven to be an efficient pedagogical catalyst. However, matching compatible users presents structural challenges. Someone seeking to master backend software development while offering native Arabic lessons needs to seamlessly find a counterpart with the exact inverse requirements. Streamify was created to formalize this process within a clean, scalable social ecosystem.

2.2 Project Scope & Vision

The baseline implementation boundaries defined for Streamify encapsulate a safe, decoupled web application. The platform provides structured profile discovery, dual-stage interactive onboarding routines, a granular relationship state manager, and secure messaging capabilities.

The engineering roadmap prioritizes decoupling the resource layers. The frontend client leverages React 19 and TanStack Query for optimal client-side caching. Meanwhile, the Node.js backend operates as a stateless API cluster ready for multi-container orchestration. This architecture ensures high availability, preventing centralized failures during concurrent connections.

3. Problem Definition

3.1 Industry Status Quo and Shortcomings

Contemporary knowledge-sharing systems suffer from structural inefficiencies that hinder real-time collaboration. The most critical challenges include:

- **Unstructured Indexing Layouts:** Forums and communities lack strict schema typing. As a result, parsing profiles for specific skill matches requires complex manual searching, leading to low conversion rates.
- **Unbounded Sub-document Overlapping:** A common database design flaw involves nesting growing arrays (like friend or pending list IDs) directly inside primary user documents. In MongoDB, this causes document size expansion toward the 16MB limit, increasing CPU overhead and lowering performance.
- **State Synchronization Bottlenecks:** Traditional HTTP polling mechanisms create high network overhead. Meanwhile, custom local WebSocket layers require significant engineering effort to ensure proper scaling, edge authentication, and room lifecycle synchronization.

3.2 The Streamify Functional Solution

Streamify addresses these architectural limitations through intentional backend engineering choices:

1. It implements strict database schemas to clearly separate stable user records from dynamic request data.
2. It structures relationships using an isolated model pattern to ensure predictable query performance.
3. It offloads messaging traffic to dedicated communication networks via programmatic token generation, keeping the primary server focused on core business logic.

4. System Architecture & Technologies

4.1 Architectural Overview Diagram

The application architecture is organized into four decoupled functional layers to ensure high modularity and seamless maintenance:

[Figure 4.1: Streamify Multi-Tier Decoupled Distributed Architecture]

React Client Layer (TanStack Query Caching)

▼ ▲ (JSON REST APIs over TLS)

Node.js / Express Application Cluster

▼ ▲ (Mongoose Connection Pooling) *▼ ▲ (Cryptographic SDK Tokens)*

MongoDB Cluster (Isolated Schemas) *Stream.io Distributed Infrastructure*

This layered approach ensures that client interactions are efficiently managed on the frontend before hitting the application cluster. Database operations benefit from dedicated connection pooling, while heavy real-time communication traffic is offloaded safely to Stream.io.

4.2 Concrete Technology Stack Definition

The framework choices were driven by performance requirements, schema compliance, and developer ecosystem support:

- **Frontend Client Infrastructure:** Built with React.js using functional components and hooks. State synchronization, automatic background refetching, and query mutations are handled via TanStack Query, minimizing unnecessary layout re-renders.
- **Backend Controller Pipeline:** Powered by Node.js and Express.js using an asynchronous router-controller architecture. Global middleware captures exceptions, ensures CORS compliance, and parses structured payloads.
- **Database Engine:** MongoDB handles document storage, leveraging Mongoose for reliable object-relational abstraction, index validation, and reference management.
- **Communication Layer:** Stream.io manages the real-time messaging pipeline. The backend uses the official server-side SDK to securely sign JWT-based tokens, authenticating users directly on the client side without routing chat data through the core API server.

5. Implementation & Dependencies

5.1 Core Database Model & Isolation Patterns

To prevent document bloat and ensure fast query times, user relationship data is decoupled from the main user account profile. Instead of keeping a list of friends inside each user document, a separate `FriendRequest` collection manages these connections.

The following Mongoose schema definition illustrates this isolated tracking pattern:

```
const mongoose = require('mongoose');

const FriendRequestSchema = new mongoose.Schema({
  senderId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
    index: true
  },
  receiverId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true,
    index: true
  },
  status: {
    type: String,
    enum: ['pending', 'accepted', 'rejected', 'blocked'],
    default: 'pending',
    required: true
  }
}, { timestamps: true });

FriendRequestSchema.index({ senderId: 1, receiverId: 1 }, { unique:
true });

module.exports = mongoose.model('FriendRequest', FriendRequestSchema);
```

5.2 Package and Module Matrix

The system relies on a clean, optimized set of production dependencies to ensure reliability and security:

PACKAGE NAME	TARGET SCOPE & ARCHITECTURAL UTILITY	RISK MANAGEMENT MITIGATION FACTOR
express	Provides HTTP pipeline routing and middleware configuration.	Configured with global exception catching to prevent application crashes.
mongoose	Manages MongoDB connections and enforces document schemas.	Utilizes connection pooling to handle concurrent database operations efficiently.
getstream	Generates authenticated communication tokens for the client-side chat interface.	Tokens are cryptographically signed server-side using secure API secret strings.
bcryptjs	Performs one-way cryptographic hashing for user credentials.	Implements secure salting values to protect passwords from brute-force exposure.
dotenv	Loads environment configurations into process runtime memory.	Keeps sensitive credentials out of source control tracking.

6. Achievements & Progress

6.1 Backend API Route Matrices

The backend API core is fully operational, featuring optimized database logic and validated routing paths across all primary modules:

HTTP VERB	RESOURCE TARGET ROUTE	BUSINESS LOGIC LAYER ACTION	DATABASE COLLECTIONS MODIFIED
POST	`/api/auth/register`	Registers new profiles with salted password hashing.	`users`
POST	`/api/auth/onboarding`	Saves user skills and locations, setting the account status to active.	`users`
POST	`/api/user/friend-request`	Creates a new connection request between two accounts.	`friendrequests`
GET	`/api/chat/token`	Generates a verified JWT access key for the chat interface.	None (Read-only cryptographic calculation)

6.2 Postman Test Scenarios & Logs

To ensure API reliability, endpoints were systematically validated using Postman integration tests. Every main logic pathway achieved passing metrics:

```
// Automated Postman Test Assertions for User Onboarding Execution
pm.test("Status code is 200 OK or 201 Created", function () {
    pm.expect(pm.response.code).to.be.oneOf([200, 201]);
});

pm.test("Profile Document shifts Onboarded status to True", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData.user.isOnboarded).to.eql(true);
    pm.expect(jsonData.user).to.have.property('skills');
});
```

- **Onboarding Validation:** Verified payload handling for geo-location data ("Egypt") along with user-defined skill parameters. The API correctly responds with a `200 OK` status code.
- **Relationship Handling:** Tested request creations to ensure matching entries are correctly saved with a `pending` status flag, enforcing unique index integrity constraints.
- **Token Delivery:** Checked server token generation to guarantee accurate payload structure and direct compatibility with frontend chat components.

7. Future Work & Gantt Plan

The project is on track according to the baseline development timeline. The primary goal for the next phase is connecting the functional backend services with the frontend user interface components.

[Figure 7.1: Project Milestone Timeline Chart]

Month 1-2: Core Backend Engine & Schema Implementation —— (100% Completed)
Month 3: API Pipeline Verification & Security Audits —— (100% Completed)
Month 4: Frontend Component Building & Integration —— (In Progress)
Month 5: Multi-Container Deployment & Cloud Orchestration —— (Planned)

Future deployment phases will focus on containerizing the application layers using Docker. This will create a clean environment for staging and production testing on AWS cloud platforms, ensuring reliable performance under realistic user traffic.

8. References

1. **MongoDB Documentation:** Architectural guidance on handling large scale document indexing and data isolation patterns. (https://docs.mongodb.com)
2. **Express Framework Manual:** Implementation patterns for building reliable middleware pipelines and handling global application exceptions. (https://expressjs.com)
3. **Stream.io Developer Reference:** Security guidelines for server-side token generation and integration with client messaging libraries. (https://getstream.io/chat/docs/)
4. **TanStack Query Guides:** Caching strategies and state synchronization techniques for asynchronous data models in React. (https://tanstack.com/query)